



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS
FACULTAD DE INGENIERÍA
PROGRAMA ACADÉMICO DE CIENCIAS DE LA COMPUTACIÓN

Aplicación Web de Soporte Clínico basada en Redes Neuronales Convolucionales para la
Identificación Automatizada de Patologías Retinianas en Consultorios de Lima Metropolitana

BENCHMARKING

AUTOR(ES)

Mayhua Hinostroza, José Antonio (202218044)

Manchay Paredes, Lucero Salome (202216120)

ASESOR(ES)

Castañeda Vargas, Pedro Segundo

Lima, 28 de Junio del 2026

INDICE

BENCHMARKING.....	3
1. INTRODUCCIÓN.....	3
2. OBJETIVO DEL BENCHMARKING.....	3
3. BENCHMARKING DE ARQUITECTURAS CNN PARA CLASIFICACIÓN DE PATOLOGÍAS RETINIANAS.....	5
3.1. Criterios y Parámetros de Medición.....	6
3.2. Comparación de Propuestas.....	8
3.3. Análisis de Alternativas.....	10
3.4. Selección de Alternativa.....	11
4. BENCHMARKING DE FRAMEWORKS DE BACKEND PARA API DE INFERENCIA CLÍNICA.....	12
3.1. Criterios y Parámetros de Medición.....	13
3.2. Comparación de Propuestas.....	16
3.3. Análisis de Alternativas.....	18
3.4. Selección de Alternativa.....	20
5. BENCHMARKING DE FRAMEWORKS DE FRONTEND PARA INTERFAZ WEB CLÍNICA.....	20
4.1. Criterios y Parámetros de Medición.....	21
4.2. Comparación de Propuestas.....	25
4.3. Análisis de Alternativas.....	27
4.4. Selección de Alternativa.....	28
6. CONCLUSIONES.....	29
7. REFERENCIAS BIBLIOGRÁFICAS.....	29

BENCHMARKING

1. INTRODUCCIÓN

La detección automatizada de patologías retinianas mediante sistemas de soporte al diagnóstico clínico (CAD) basados en inteligencia artificial representa una alternativa tecnológica relevante frente a la saturación de los consultorios oftalmológicos en Lima Metropolitana. Enfermedades como la retinopatía diabética y el glaucoma pueden avanzar de forma progresiva y silenciosa, por lo que la identificación temprana es fundamental para priorizar la atención de pacientes con riesgo visual. En este contexto, las redes neuronales convolucionales (CNN) aplicadas al análisis de imágenes de fondo de ojo permiten apoyar la clasificación inicial de casos, brindando al especialista una segunda opinión objetiva y reduciendo la dependencia del análisis manual.

Sin embargo, las soluciones existentes no se ajustan completamente al problema delimitado en este trabajo. Diversos estudios muestran el potencial de los modelos de aprendizaje profundo para detectar enfermedades retinianas, como los propuestos por Ejaz et al. (2024), Henge et al. (2025) y Arias-Serrano et al. (2024). No obstante, muchas de estas soluciones han sido entrenadas y validadas con bases de datos internacionales, sin considerar las condiciones operativas, tecnológicas y organizacionales de los consultorios peruanos. En el contexto local, los procesos clínicos pueden depender de flujos parcialmente analógicos, almacenamiento fragmentado e interoperabilidad limitada entre sistemas, lo que dificulta la adopción directa de soluciones externas (Infante & Cruzado, 2024).

Asimismo, una solución CAD no debe limitarse al modelo predictivo. Para operar en un entorno clínico real, necesita integrarse con una arquitectura web segura, un backend capaz de procesar imágenes e invocar el servicio de inferencia con baja latencia, y una interfaz clínica comprensible para el personal técnico y médico. Además, el sistema debe considerar la protección de datos personales y clínicos bajo la Ley N.º 29733, así como la necesidad de operar sin instalar servidores locales adicionales en el consultorio.

Por estas razones, resulta necesario realizar un benchmarking tecnológico de los componentes principales de la solución propuesta. Este análisis compara tres elementos centrales: las arquitecturas CNN para la clasificación de imágenes retinianas, los frameworks

de backend para exponer el servicio de inferencia mediante una API clínica, y los frameworks de frontend para construir una interfaz web segura, mantenible y usable. La finalidad del benchmarking no es seleccionar la tecnología más popular, sino identificar las alternativas que mejor respondan a los requerimientos del sistema: precisión diagnóstica, integración con procesamiento de imágenes médicas, baja latencia, compatibilidad con despliegue en la nube, seguridad de datos clínicos y facilidad de adopción en el flujo de atención oftalmológica.

2. OBJETIVO DEL BENCHMARKING

El presente benchmarking tiene como objetivo evaluar y seleccionar las tecnologías más adecuadas para los tres componentes principales del modelo tecnológico de la Aplicación Web de Soporte Clínico basada en CNN. El análisis se orienta a comparar arquitecturas, algoritmos y frameworks que permitan implementar una solución web capaz de procesar imágenes de fondo de ojo, ejecutar inferencia mediante inteligencia artificial y presentar resultados diagnósticos de forma segura y comprensible para el personal médico. Esta evaluación se apoya en estudios previos que muestran la aplicabilidad de las CNN y del Transfer Learning en la detección de enfermedades retinianas (Ejaz et al., 2024; Shamrat et al., 2024; Mamyrbayev et al., 2025; Rahim et al., 2024).

La finalidad del benchmarking es sustentar técnicamente la selección de los componentes que conformarán la solución propuesta, considerando criterios como desempeño diagnóstico, compatibilidad con procesamiento de imágenes médicas, integración con servicios web, seguridad, mantenibilidad, escalabilidad y facilidad de adopción en consultorios oftalmológicos de Lima Metropolitana. Para ello, la evaluación se apoya en la literatura científica revisada durante el estado del arte, especialmente en estudios sobre redes neuronales convolucionales, sistemas CAD, aplicaciones clínicas basadas en inteligencia artificial y arquitecturas web para soluciones de salud digital (Moreno et al., 2024; Ticlavilca et al., 2024; Infante & Cruzado, 2024).

Los tres componentes evaluados son:

- **Componente 1: Arquitecturas de Redes Neuronales Convolucionales (CNN).**

Este componente evalúa alternativas de modelos CNN para la clasificación automatizada

de patologías retinianas, específicamente glaucoma, retinopatía diabética y casos sin hallazgos patológicos, a partir de imágenes de fondo de ojo. La comparación considera el desempeño diagnóstico, la cobertura clínica, la capacidad de generalización, la compatibilidad con Transfer Learning y la eficiencia de inferencia para un entorno de triaje oftalmológico (Ejaz et al., 2024; Shamrat et al., 2024; Mamyrbayev et al., 2025; Rahim et al., 2024).

- **Componente 2: Frameworks de backend para API de inferencia clínica.**

Este componente compara frameworks backend capaces de recibir imágenes desde la interfaz web, validar los datos enviados, comunicarse con el modelo CNN y devolver resultados probabilísticos mediante una API REST. La evaluación considera la compatibilidad con el ecosistema de inteligencia artificial en Python, el rendimiento, la documentación automática de servicios, la seguridad y la velocidad de desarrollo, tomando como referencia tecnologías usadas en soluciones de inferencia y despliegue web de modelos de aprendizaje profundo (tiangolo, 2026; TensorFlow, 2021; Ticlavilca et al., 2024; Moreno & Ticlavilca, 2025).

- **Componente 3: Frameworks de frontend para interfaz web clínica.**

Este componente evalúa frameworks frontend orientados a construir una interfaz web accesible para el personal técnico y médico del Centro Oftalmológico V.E.R. La comparación considera la arquitectura del framework, la seguridad en la interacción con datos clínicos, la integración con APIs REST, la experiencia de usuario, la accesibilidad y el soporte de tipado estático para reducir errores durante el desarrollo, criterios relacionados con la necesidad de construir sistemas web clínicos seguros, mantenibles y centrados en el usuario (Ticlavilca et al., 2024; Moreno & Ticlavilca, 2025; ISO/IEC 25010, 2023; Ley N.º 29733, 2011).

3. BENCHMARKING DE ARQUITECTURAS CNN PARA CLASIFICACIÓN DE PATOLOGÍAS RETINIANAS

Este componente compara alternativas de arquitectura CNN para el motor de clasificación de imágenes de fondo de ojo. Dado que la tesis propone una aplicación web CAD para apoyar la detección temprana de glaucoma y retinopatía diabética en consultorios de Lima Metropolitana, la alternativa elegida debe equilibrar desempeño diagnóstico, cobertura clínica, compatibilidad con transfer learning, eficiencia de inferencia y posibilidad de despliegue en una arquitectura web o cloud.

A continuación, se plantean las propuestas que serán parte de la evaluación:

Tabla 1

Propuestas seleccionadas

Propuesta	Descripción	Fuente (cita en APA 7)
DenseNet121 con Transfer Learning	Arquitectura CNN con conexiones densas evaluada para la detección temprana de múltiples enfermedades retinianas sobre ODIR.	Ejaz et al. (2024)
DRNet13	Red neuronal profunda de 13 capas diseñada para el análisis de imágenes de fondo de ojo en retinopatía diabética.	Shamrat et al. (2024)
CNN híbrida CNN+RNN	Modelo híbrido que combina redes convolucionales con redes recurrentes para capturar patrones espaciales y relaciones vasculares en retinografías.	Mamyrbayev et al. (2025)

CNN multicapa con Dropout	Arquitectura multicapa orientada a la clasificación de glaucoma con regularización mediante dropout y ajuste de filtros y épocas.	Rahim et al. (2024)
----------------------------------	---	---------------------

3.1. Criterios y Parámetros de Medición

Tabla 2

Definición de criterios y parámetros de medición

#	Criterio	Descripción	Unidad	Valores	Parámetro de Medición (*)	Fuente (cita en APA 7)
1	Desempeño diagnóstico	Capacidad del modelo para clasificar correctamente imágenes retinianas considerando accuracy, sensibilidad, F1-score o AUC-ROC.	Escala	[1 – 5]	1 = < 85% 2 = 85%-89% 3 = 90%-93% 4 = 94%-96% 5 = > 96% en métricas reportadas	Ejaz et al. (2024); Shamrat et al. (2024)
2	Cobertura clínica	Nivel de adecuación para detectar más de una patología retiniana o soportar clasificación multiclase.	Escala	[1 – 5]	1 = patología no retiniana 2 = una sola patología 3 = dos patologías 4 = varias clases relacionadas	Ejaz et al. (2024); Rahim et al. (2024)

					5 = múltiples enfermedades retinianas.	
3	Generalización y validación	Solidez de la evidencia experimental según datasets, aumento de datos y validación.	Escala	[1 – 5]	1 = validación limitada 2 = un dataset pequeño 3 = dataset público 4 = varios datasets o aumento de datos 5 = validación multidataset robusta y control de incertidumbre.	Shamrat et al. (2024); Kumar et al. (2026)
4	Transfer Learning y despliegue web	Disponibilidad de pesos preentrenados, facilidad de fine-tuning e implementación en una API Python/TensorFlow-Keras o servicio cloud.	Escala	[1 – 5]	1 = sin soporte de TL o no portable 2 = entorno específico 3 = portable con adaptación 4 = compatible con Python 5 = compatible directamente con TensorFlow/Keras y servicios web.	TensorFlow (2021); Ticlavilca-Inche et al. (2024)
5	Eficiencia de inferencia para triaje	Aptitud para procesar retinografías con baja latencia y consumo	Escala	[1 – 5]	1 = alta carga computacional 2 = requiere hardware especializado	Mamyrbayev et al. (2025); Shamrat et al. (2024)

		computacional razonable en consultorios.			3 = eficiencia media 4 = apto para inferencia web 5 = baja latencia y arquitectura ligera.	
--	--	--	--	--	---	--

(*) Se debe realizar una parametrización para que los valores diferentes a escala puedan definirse dentro de los valores del 1 al 5.

3.2. Comparación de Propuestas

Tabla 3

Matriz de enfrentamiento

	Criterio	A	B	C	D	E	Subtotal	Ponderación
A	Desempeño diagnóstico	-	1	1	1	0	3	30%
B	Cobertura clínica	0	-	1	0	1	2	20%
C	Generalización y validación	0	0	-	1	1	2	20%
D	Transfer Learning y despliegue web	0	1	0	-	1	2	20%
E	Eficiencia de inferencia para triaje	1	0	0	0	-	1	10%
Total							10	100%

(0,1) En este campo solo se puede colocar valores de 0 y 1, donde 0 quiere decir que es menos importante que otro criterio y 1 hace énfasis a que este criterio es más importante que el criterio con el cual se compara. Ejemplo: Si consideramos que el Criterio 1(A) es menos importante que el Criterio 2 (B) entonces tendríamos que colocar 0 en la intersección entre A y B

Tabla 4

Matriz de Priorización

Criterio	Impacto	Arquitecturas CNN para clasificación de patologías retinianas							
		DenseNet121		DRNet13		CNN+RNN		CNN Dropout	
		Puntaje	Promedio	Puntaje	Promedio	Puntaje	Promedio	Puntaje	Promedio
Desempeño diagnóstico	30%	4	1.20	5	1.50	4	1.20	5	1.50
Cobertura clínica	20%	5	1.00	3	0.60	4	0.80	2	0.40
Generalización y validación	20%	4	0.80	4	0.80	3	0.60	2	0.40
Transfer Learning y despliegue web	20%	5	1.00	3	0.60	3	0.60	2	0.40

Eficiencia de inferencia para triaje	10%	4	0.40	4	0.40	3	0.30	3	0.30
Promedio Total	100%		4.40		3.90		3.50		3.00

[1 – 5] Valor del 1 al 5 que ha sido establecido en la tabla de parámetros de medición.

Impacto. Este % se obtiene de la columna Ponderación de la Matriz de Enfrentamiento

3.3. Análisis de Alternativas

Tabla 5

Cuadro de Análisis de Alternativas por Criterio

Criterio	Impacto	Evaluación
Desempeño diagnóstico	30%	DRNet13 y la CNN multicapa con Dropout obtienen los puntajes más altos por sus métricas superiores al 97% y 99% respectivamente. DenseNet121 mantiene una calificación alta por sus métricas cercanas al 96% en un escenario multiclase.
Cobertura clínica	20%	DenseNet121 lidera por haber sido aplicado a la detección de múltiples enfermedades retinianas. La CNN con Dropout queda rezagada porque se concentra en glaucoma binario, aunque sus métricas sean sobresalientes.
Generalización y validación	20%	DRNet13 destaca por usar APTOS 2019, IDRiD y EyePACS. DenseNet121 es competitivo por su uso de ODIR y aumento de datos, mientras que CNN+RNN y CNN con Dropout requieren mayor validación clínica externa.

Transfer Learning y despliegue web	20%	DenseNet121 obtiene el mayor puntaje porque puede integrarse de forma natural con TensorFlow/Keras, fine-tuning y una API Python. La alternativa de Rahim et al. queda limitada por su implementación en MATLAB.
Eficiencia de inferencia para triaje	10%	DRNet13 y DenseNet121 son viables para un servicio de inferencia web. CNN+RNN podría requerir mayor procesamiento por la fusión secuencial, aunque mantiene utilidad en el análisis de patrones vasculares.

3.4. Selección de Alternativa

De acuerdo con el análisis realizado, la arquitectura más adecuada para el motor de clasificación es DenseNet121 con Transfer Learning, con un puntaje promedio de 4.40. Esta elección se justifica porque presenta un desempeño diagnóstico alto, está orientada a la detección multiclase de patologías retinianas y puede integrarse con TensorFlow/Keras dentro de una API Python. Aunque DRNet13 y la CNN con Dropout reportan métricas superiores en escenarios específicos, DenseNet121 ofrece una relación más equilibrada entre precisión, cobertura clínica, reproducibilidad, adaptación mediante fine-tuning y despliegue web/cloud para el contexto de los consultorios oftalmológicos de Lima Metropolitana.

4. BENCHMARKING DE FRAMEWORKS DE BACKEND PARA API DE INFERENCIA CLÍNICA

Este componente evalúa frameworks backend para recibir retinografías desde el navegador, validar datos clínicos, invocar el modelo CNN como servicio de inferencia y devolver resultados probabilísticos al personal médico. La selección es importante porque el backend debe integrarse con el ecosistema Python de inteligencia artificial, soportar documentación de API, ofrecer baja latencia para inferencias concurrentes y mantener controles de seguridad sobre información sensible de pacientes.

A continuación, se plantean las propuestas que serán parte de la evaluación:

Tabla 1

Propuestas seleccionadas

Propuesta	Descripción	Fuente (cita en APA 7)
FastAPI	Framework web asíncrono de Python basado en Starlette y Pydantic. Permite construir APIs REST de alto rendimiento con validación de datos, documentación OpenAPI/Swagger automática y compatibilidad directa con TensorFlow, Keras y OpenCV para servir modelos CNN.	tiangolo (2026); FastAPI (s.f.)
Flask	Microframework WSGI de Python, ligero y flexible. Ha sido usado para exponer servicios de inferencia de modelos CNN mediante endpoints REST, aunque requiere más configuración manual para validación, seguridad y documentación.	Pallets Projects (s.f.); Moreno Lozano & Ticlavilca Inche (2025)
Django	Framework web de alto nivel en Python con ORM, administración integrada y medidas de seguridad robustas. Es adecuado para sistemas completos y puede complementarse con Django REST Framework, pero añade complejidad para una API de inferencia liviana.	Django Software Foundation (s.f.)
Spring Boot	Framework Java para aplicaciones empresariales y microservicios. Ofrece robustez, rendimiento y seguridad, pero su integración	Spring (s.f.); Moreno Lozano & Ticlavilca Inche (2025)

	directa con modelos de IA en Python requiere servicios puente o separación adicional de componentes.	
--	--	--

3.1. Criterios y Parámetros de Medición

Tabla 2

Definición de criterios y parámetros de medición

#	Criterio	Descripción	Unidad	Valores	Parámetro de Medición (*)	Fuente (cita en APA 7)
1	Compatibilidad con ecosistema IA	Facilidad para integrar TensorFlow/Keras, OpenCV, scikit-learn y scripts de inferencia para el procesamiento de imágenes de retina.	Escala	1-5	1 = integración indirecta y compleja con librerías de IA. 2 = integración posible mediante adaptadores o servicios externos. 3 = integración funcional con configuración adicional. 4 = integración directa con librerías de IA, aunque requiere ciertos ajustes. 5 = integración nativa en Python con TensorFlow/Keras, OpenCV y scikit-learn sin dependencias adicionales complejas.	TensorFlow (2021); tiangolo (2026)

2	Rendimiento y latencia	Capacidad de atender múltiples solicitudes concurrentes de inferencia CNN sin afectar la experiencia clínica.	Escala	1-5	1 = latencia alta o baja capacidad de concurrencia. 2 = rendimiento aceptable solo para cargas ligeras. 3 = rendimiento medio para solicitudes estándar. 4 = buen rendimiento bajo concurrencia moderada. 5 = alto rendimiento para APIs concurrentes/asíncronas de inferencia clínica.	FastAPI (s.f.); Spring (s.f.)
3	Documentación automática y estándares API	Soporte para documentación de endpoints, esquemas, OpenAPI/Swagger y comunicación REST con sistemas externos.	Escala	1-5	1 = documentación manual sin generación automática. 2 = documentación básica mediante configuración externa. 3 = soporte parcial para documentación de endpoints. 4 = generación documentada de esquemas y endpoints con configuración moderada.	FastAPI (s.f.); tiangolo (2026)

					5 = documentación automática OpenAPI/Swagger con esquemas claros y mantenibles.	
4	Seguridad y mantenibilidad clínica	Capacidad para estructurar autenticación, validación, control de vulnerabilidades web y mantenimiento del servicio clínico.	Escala	1-5	1 = seguridad dependiente casi totalmente del desarrollador. 2 = seguridad básica mediante extensiones o configuración manual. 3 = seguridad configurable para autenticación, validación y control de acceso. 4 = seguridad robusta con validación de datos, autenticación y estructura mantenible. 5 = seguridad y mantenibilidad robustas para sistemas clínicos, con controles claros de autenticación, validación, estructura modular y buenas prácticas.	Django Software Foundation (s.f.); Spring (s.f.); OWASP Foundation (s.f.)
5	Velocidad de desarrollo	Rapidez para construir, probar y desplegar endpoints	Escala	1-5	1 = desarrollo lento con alta configuración inicial. 2 = desarrollo moderadamente lento por boilerplate o dependencias adicionales.	tiangolo (2026); Pallets Projects (s.f.)

		funcionales para el prototipo CAD.			3 = velocidad media para construir endpoints funcionales. 4 = desarrollo ágil con herramientas de validación y estructura clara. 5 = desarrollo rápido con bajo boilerplate, tipado, validación automática y documentación integrada.	
--	--	------------------------------------	--	--	--	--

3.2. Comparación de Propuestas

Tabla 3

Matriz de enfrentamiento

	Criterio	A	B	C	D	E	Subtotal	Ponderación
A	Compatibilidad con ecosistema IA	-	1	1	1	0	3	30%
B	Rendimiento y latencia	0	-	1	0	1	2	20%
C	Documentación automática y estándares API	0	0	-	1	1	2	20%
D	Seguridad y mantenibilidad clínica	0	1	0	-	1	2	20%

E	Velocidad de desarrollo	1	0	0	0	-	1	10%
Total							10	100%

(0,1) En este campo solo se puede colocar valores de 0 y 1, donde 0 quiere decir que es menos importante que otro criterio y 1 hace énfasis a que este criterio es más importante que el criterio con el cual se compara. Ejemplo: Si consideramos que el Criterio 1(A) es menos importante que el Criterio 2 (B) entonces tendríamos que colocar 0 en la intersección entre A y B

Tabla 4

Matriz de Priorización

Criterio	Impacto	Frameworks de backend para API de inferencia clínica							
		FastAPI		Flask		Django		Spring Boot	
		Puntaje	Promedio	Puntaje	Promedio	Puntaje	Promedio	Puntaje	Promedio
Compatibilidad con ecosistema IA	30%	5	1.50	5	1.50	4	1.20	2	0.60
Rendimiento y latencia	20%	5	1.00	3	0.60	3	0.60	5	1.00
Documentación	20%	5	1.00	3	0.60	4	0.80	4	0.80

automática y estándares API									
Seguridad y mantenibilidad clínica	20%	4	0.80	3	0.60	5	1.00	5	1.00
Velocidad de desarrollo	10%	5	0.50	4	0.40	3	0.30	3	0.30
Promedio Total	100%		4.80		3.70		3.90		3.70

[1 – 5] Valor del 1 al 5 que ha sido establecido en la tabla de parámetros de medición.

Impacto. Este % se obtiene de la columna Ponderación de la Matriz de Enfrentamiento

3.3. Análisis de Alternativas

Tabla 5

Cuadro de Análisis de Alternativas por Criterio

Criterio	Impacto	Evaluación
Compatibilidad con ecosistema IA	30%	FastAPI y Flask alcanzan el puntaje máximo por ejecutarse en Python y facilitar la conexión directa con TensorFlow/Keras y OpenCV. Spring Boot queda por debajo porque requiere desacoplar el modelo en un servicio adicional o usar adaptadores.

Rendimiento y latencia	20%	FastAPI y Spring Boot son las alternativas más sólidas para concurrencia y producción. FastAPI destaca por su enfoque async/await para APIs de inferencia; Flask y Django pueden responder de forma adecuada, pero no son la opción más especializada para cargas concurrentes.
Documentación automática y estándares API	20%	FastAPI lidera por generar documentación OpenAPI y esquemas de forma automática. Django y Spring Boot son robustos, aunque suelen requerir configuración adicional para APIs documentadas. Flask queda por detrás al depender de extensiones o documentación manual.
Seguridad y mantenibilidad clínica	20%	Django y Spring Boot destacan por su madurez empresarial y controles integrados. FastAPI mantiene un puntaje alto por su validación estricta de datos y soporte para esquemas de autenticación, aunque ciertos controles deben implementarse de forma explícita.
Velocidad de desarrollo	10%	FastAPI obtiene el mayor puntaje por su bajo boilerplate, tipado y documentación automática, lo que reduce el esfuerzo para construir endpoints de carga de imagen, inferencia y consulta de resultados.

3.4. Selección de Alternativa

De acuerdo con el análisis realizado, el framework backend más adecuado es FastAPI, con un puntaje promedio de 4.80. Esta elección se justifica por su compatibilidad nativa con Python, TensorFlow/Keras y OpenCV, su arquitectura asíncrona para servicios de inferencia, la validación de datos mediante Pydantic y la generación automática de documentación OpenAPI. Para una

aplicación CAD de soporte clínico, FastAPI permite construir una API REST clara, mantenible y alineada con el flujo de carga de imagen, preprocesamiento, inferencia y retorno de resultados diagnósticos, sin introducir capas adicionales innecesarias

5. **BENCHMARKING DE FRAMEWORKS DE FRONTEND PARA INTERFAZ WEB CLÍNICA**

Este componente compara frameworks frontend para desarrollar la interfaz web que usarán técnicos, optometristas y oftalmólogos. La interfaz debe permitir el registro de casos, la carga de retinografías, la visualización de resultados probabilísticos del modelo, la trazabilidad del análisis y la navegación segura. Dado que se trabajará con información clínica y flujos de triaje, se priorizan la arquitectura empresarial, la seguridad frente a vulnerabilidades web, la integración con APIs REST, la experiencia de usuario y el tipado estático para reducir errores en un sistema de soporte diagnóstico.

A continuación, se plantean las propuestas que serán parte de la evaluación:

Tabla 1
Propuestas seleccionadas

Propuesta	Descripción	Fuente (cita en APA 7)
Angular	Framework web de Google basado en TypeScript, con componentes, inyección de dependencias, enrutamiento, Angular CLI, HttpClient, interceptores y sanitización frente a XSS. Es adecuado para aplicaciones empresariales y clínicas mantenibles.	Angular (s.f.); Microsoft (2024)
React	Biblioteca para construir interfaces web mediante componentes reutilizables y un ecosistema amplio. Ofrece flexibilidad y buen soporte para TypeScript, aunque requiere seleccionar librerías	Meta Open Source (s.f.)

	externas para arquitectura, formularios, estado y seguridad complementaria.	
Vue.js	Framework progresivo para interfaces web, adoptable de forma incremental y con una curva de aprendizaje baja. Es versátil para prototipos y aplicaciones medianas, aunque su estandarización empresarial depende más de las decisiones del equipo.	Vue.js (s.f.)
Flutter Web	Toolkit multiplataforma basado en Dart que permite crear aplicaciones web, móviles y de escritorio desde una misma base de código. Es útil cuando se prioriza la multiplataforma, pero puede no ser la opción más adecuada para una aplicación web clínica basada en formularios, rutas y APIs REST.	Ticlavilca-Inche et al. (2024); Moreno Lozano & Ticlavilca Inche (2025)

4.1. Criterios y Parámetros de Medición

Tabla 2

Definición de criterios y parámetros de medición

#	Criterio	Descripción	Unidad	Valores	Parámetro de Medición (*)	Fuente (cita en APA 7)
1	Escalabilidad y arquitectura	Capacidad para sostener una aplicación clínica con	Escala	[1 – 5]	1 = estructura limitada sin modularización clara.	Angular (s.f.); Meta Open Source (s.f.)

		módulos de pacientes, carga de imágenes, resultados de IA e historial sin afectar la mantenibilidad.			2 = estructura básica con componentes reutilizables, pero con organización manual. 3 = modularidad media con componentes y rutas organizadas. 4 = arquitectura sólida con separación de responsabilidades, servicios, rutas y módulos. 5 = arquitectura robusta con módulos, servicios, rutas, inyección de dependencias, lazy loading y alta mantenibilidad.	
2	Seguridad integrada	Herramientas para reducir vulnerabilidades comunes de aplicaciones web y proteger datos clínicos sensibles.	Escala	[1 – 5]	1 = la seguridad depende totalmente del desarrollador. 2 = protecciones básicas mediante validaciones o librerías externas. 3 = controles parciales para sanitización, autenticación y manejo seguro de rutas.	Angular (s.f.); OWASP Foundation (s.f.)

					4 = mecanismos documentados para reducir riesgos en vistas, formularios, tokens y comunicación con APIs. 5 = mecanismos integrados y buenas prácticas documentadas para proteger datos sensibles, rutas, sesiones y consumo seguro de servicios.	
3	Integración con APIs REST	Facilidad para consumir endpoints de autenticación, carga de imágenes, inferencia clínica y consulta de resultados.	Escala	[1 – 5]	1 = integración compleja y con configuración extensa. 2 = integración básica mediante código repetitivo o dependencias adicionales. 3 = integración estándar con consumo HTTP estable. 4 = integración estructurada mediante servicios, manejo de errores e interceptores o equivalentes. 5 = integración simple, tipable y mantenible con servicios, interceptores, manejo de tokens y comunicación clara con el backend.	Angular (s.f.); FastAPI (s.f.)

4	UX y accesibilidad	Disponibilidad de patrones, componentes y prácticas para construir interfaces intuitivas para el personal médico.	Escala	[1 – 5]	1 = sin componentes UI reutilizables. 2 = componentes básicos con personalización limitada. 3 = ecosistema moderado de componentes visuales. 4 = ecosistema amplio con librerías maduras, diseño responsive y soporte de accesibilidad. 5 = ecosistema amplio, accesible y con patrones maduros para interfaces clínicas mantenibles.	Angular (s.f.); Vue.js (s.f.)
5	Soporte de TypeScript y tipado	Nivel de soporte para tipado estático que reduzca errores en tiempo de compilación en un sistema clínico.	Escala	[1 – 5]	1 = sin tipado estático. 2 = soporte mínimo mediante configuración adicional. 3 = soporte opcional de TypeScript. 4 = soporte oficial y ampliamente integrado. 5 = TypeScript como base del framework o tipado fuertemente integrado por defecto.	Angular (s.f.); Meta Open Source (s.f.)

(*) Se debe realizar una parametrización para que los valores diferentes a escala puedan definirse dentro de los valores del 1 al 5.

4.2. Comparación de Propuestas

Tabla 3

Matriz de enfrentamiento

	Criterio	A	B	C	D	E	Subtotal	Ponderación
A	Escalabilidad y arquitectura	-	1	1	1	1	4	35%
B	Seguridad integrada	0	-	1	1	1	3	25%
C	Integración con APIs REST	0	0	-	1	1	2	20%
D	UX y accesibilidad	0	0	0	-	1	1	15%
E	Soporte de TypeScript y tipado	0	0	0	0	-	0	5%
Total							10	100%

(0,1) En este campo solo se puede colocar valores de 0 y 1, donde 0 quiere decir que es menos importante que otro criterio y 1 hace énfasis a que este criterio es más importante que el criterio con el cual se compara. Ejemplo: Si consideramos que el Criterio 1(A) es menos importante que el Criterio 2 (B) entonces tendríamos que colocar 0 en la intersección entre A y B

Tabla 4

Matriz de Priorización

Criterio	Impacto	Frameworks de frontend para interfaz web clínica							
		Angular		React		Vue.js		Flutter Web	
		Puntaje	Promedio	Puntaje	Promedio	Puntaje	Promedio	Puntaje	Promedio
Escalabilidad y arquitectura	35%	5	1.75	4	1.40	3	1.05	3	1.05
Seguridad integrada	25%	5	1.25	3	0.75	3	0.75	3	0.75
Integración con APIs REST	20%	5	1.00	5	1.00	4	0.80	3	0.60
UX y accesibilidad	15%	4	0.60	4	0.60	3	0.45	3	0.45
Soporte de TypeScript y tipado	5%	5	0.25	3	0.15	3	0.15	4	0.20
Promedio Total	100%		4.85		3.90		3.20		3.05

[1 – 5] Valor del 1 al 5 que ha sido establecido en la tabla de parámetros de medición.

Impacto. Este % se obtiene de la columna Ponderación de la Matriz de Enfrentamiento

4.3. Análisis de Alternativas

Tabla 5

Cuadro de Análisis de Alternativas por Criterio

Criterio	Impacto	Evaluación
Escalabilidad y arquitectura	35%	Angular obtiene el mayor puntaje por su estructura de framework completo, con módulos, componentes, servicios, rutas e inyección de dependencias. React es competitivo, pero requiere decisiones arquitectónicas adicionales. Vue.js y Flutter Web pueden escalar, aunque con menor estandarización para este tipo de sistema clínico.
Seguridad integrada	25%	Angular lidera por sus mecanismos documentados de sanitización y protección frente a XSS. React, Vue.js y Flutter Web pueden ser seguros, pero dependen más de librerías, configuración y disciplina del equipo.
Integración con APIs REST	20%	Angular y React empatan en su capacidad para consumir APIs REST. Angular destaca por HttpClient, interceptores y servicios tipados; React lo hace por su flexibilidad y ecosistema. Vue.js es sólido con librerías externas, mientras que Flutter Web requiere una integración menos natural para una aplicación web clínica tradicional.
UX y accesibilidad	15%	Angular y React cuentan con ecosistemas maduros de componentes y patrones de interfaz. Angular Material y su documentación facilitan la construcción de formularios, tablas y navegación para flujos clínicos; Vue.js y Flutter Web tienen opciones útiles, pero menos alineadas con una aplicación web empresarial académica.

Soporte de TypeScript y tipado	5%	Angular lidera porque TypeScript es parte central de su arquitectura. React y Vue.js pueden usar TypeScript, pero su adopción depende del equipo. Flutter Web usa Dart, lo que ofrece tipado aunque se aleja del stack TypeScript seleccionado para el proyecto.
---------------------------------------	-----------	--

4.4. Selección de Alternativa

De acuerdo con el análisis realizado, el framework frontend más adecuado es Angular, con un puntaje promedio de 4.85. Esta elección se justifica por su arquitectura modular empresarial, el uso de TypeScript como base, la inyección de dependencias, el enrutamiento, las herramientas CLI, los servicios tipados, los interceptores HTTP y los mecanismos de sanitización frente a XSS. Para la aplicación web de soporte clínico, Angular permite construir una interfaz robusta para la carga de retinografías, la visualización de resultados probabilísticos y la gestión de casos con mayor consistencia arquitectónica que alternativas más flexibles pero menos prescriptivas.

6. CONCLUSIONES

El benchmarking confirma que la solución debe integrar tecnologías orientadas específicamente al diagnóstico asistido por inteligencia artificial, no solo componentes web genéricos. La naturaleza clínica del problema exige precisión diagnóstica, trazabilidad, interoperabilidad, seguridad de datos personales y tiempos de respuesta adecuados para el triaje oftalmológico.

Para el componente algorítmico se selecciona DenseNet121 con Transfer Learning, por su balance entre desempeño, cobertura multiclase, capacidad de fine-tuning y compatibilidad con una arquitectura Python/TensorFlow. Esta alternativa es coherente con el objetivo de clasificar patologías retinianas desde imágenes de fondo de ojo y apoyar la decisión médica sin reemplazar al especialista.

Para el backend se selecciona FastAPI, ya que permite exponer el modelo CNN mediante servicios REST de alto rendimiento, con validación de datos mediante Pydantic, documentación OpenAPI automática y compatibilidad directa con el ecosistema de Machine Learning en Python.

Para el frontend se selecciona Angular, por su estructura empresarial, seguridad integrada, componentización, soporte TypeScript y facilidad para organizar servicios que consuman la API clínica. Esta elección fortalece la mantenibilidad de la plataforma web y facilita la construcción de pantallas de carga de imágenes, visualización de resultados y gestión de casos.

En conjunto, la combinación DenseNet121 + FastAPI + Angular constituye una base tecnológica coherente para una aplicación web CAD escalable, segura y mantenible, alineada con los requerimientos del caso de estudio en consultorios oftalmológicos de Lima Metropolitana.

7. REFERENCIAS BIBLIOGRÁFICAS

- Angular. (s.f.). What is Angular? <https://angular.dev/overview>
- Arias-Serrano, I., Velásquez-López, P. A., Avila-Briones, L. N., Laurido-Mora, F. C., Villalba-Meneses, F., Tirado-Espin, A., Cruz-Varela, J., & Almeida-Galárraga, D. (2024). Artificial intelligence based glaucoma and diabetic retinopathy detection using MATLAB - retrained AlexNet convolutional neural network. F1000Research, 12, 14. <https://f1000research.com/articles/12-14/v2>
- Django Software Foundation. (s.f.). Django documentation. <https://docs.djangoproject.com/>
- Ejaz, S., Baig, R., Ashraf, Z., Alnfai, M. M., Alnahari, M. M., & Alotaibi, R. M. (2024). A deep learning framework for the early detection of multi-retinal diseases. PLOS ONE, 19(7), e0307317. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0307317>
- FastAPI. (s.f.). FastAPI framework, high performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com/>
- Flutter. (s.f.). Build apps for any screen. <https://flutter.dev/>
- Henge, S. K., Viraati, N. R., Alhussein, M., Kushwaha, A. S., Aurangzeb, K., & Singh, R. (2025). Detection of diabetic retinopathy using a multi-decision Inception-ResNet-blended hybrid model. IEEE Access, 13, 8992-9005. <https://ieeexplore.ieee.org/document/10820183>
- Infante Cueva, W., & Cruzado Benites, P. M. S. (2024). Desarrollo de un sistema de diagnóstico en la detección temprana de cataratas utilizando redes neuronales convolucionales [Tesis de título profesional, Universidad Privada Antenor Orrego]. Repositorio de la Universidad Privada Antenor Orrego. <https://repositorio.upao.edu.pe/item/2f51ef45-e2d6-4b7f-9a94-2b818eb63b91>

- International Organization for Standardization and International Electrotechnical Commission. (2023). Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model (Norma ISO/IEC No. 25010:2023). <https://www.iso.org/standard/78176.html>
- Ley 29733. (2011). Ley de protección de datos personales. Congreso de la República del Perú.
- Mamyrbayev, O., Pavlov, S., Poplavskyi, O., Momynzhanova, K., Saldan, Y., Zhanegiz, A., Zhumagulova, S., & Zhumazhan, N. (2025). Hybrid Neural Architectures Combining Convolutional and Recurrent Networks for the Early Detection of Retinal Pathologies. Engineering, Technology & Applied Science Research, 15(4), 25150–25157. <https://scopus.upc.elogim.com/pages/publications/105013113627?origin=document-preview-flyout>
- Meta Open Source. (s.f.). React. <https://react.dev/>
- Moreno Lozano, M. I., & Ticlavilca Inche, E. J. (2025). Solución móvil para la detección de Pterigión utilizando un modelo de clasificación basado en la arquitectura de CNN ResNext50 en imágenes del segmento anterior del ojo para el Centro Peruano del Pterigión [Tesis de título profesional, Universidad Peruana de Ciencias Aplicadas]. Repositorio Académico UPC. <https://repositorioacademico.upc.edu.pe/handle/10757/688399>
- Moreno-Lozano, M. I., Ticlavilca-Inche, E. J., Mauricio, D., & Oñate-Andino, A. (2024). A Performance Evaluation of Convolutional Neural Network Architectures for Pterygium Detection in Anterior Segment Eye Images. Diagnostics, 14(18), 2026. <https://www.mdpi.com/2075-4418/14/18/2026>
- OWASP Foundation. (s.f.). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- Pallets Projects. (s.f.). Welcome to Flask. <https://flask.palletsprojects.com/>
- Rahim, M. J., Al-Zuky, A. A. D., & Al-Obaidi, F. E. M. A. (2024). Glaucoma disease classification based on convolutional neural network architecture method. Mustansiriyah

- Shamrat, F. M. J. M., Shakil, R., Sharmin, Hoque Ovy, N., Akter, B., Ahmed, M. Z., Ahmed, K., Bui, F. M., & Moni, M. A. (2024). An advanced deep neural network for fundus image analysis and enhancing diabetic retinopathy detection. Healthcare Analytics, 5, 100303.
<https://www.sciencedirect.com/science/article/pii/S2772442524000054?via%3Dihub>
- Spring. (s.f.). Spring Boot. <https://spring.io/projects/spring-boot>
- TensorFlow. (2021, 24 de marzo). TensorFlow Core. <https://www.tensorflow.org/guide?hl=es-419>
- tiangolo. (2026). FastAPI [Software]. <https://fastapi.tiangolo.com>
- Ticlavilca-Inche, E. J., Moreno-Lozano, M. I., Castañeda, P., Wong-Durand, S., & Oñate-Andino, A. (2024). Mobile Application Based on Convolutional Neural Networks for Pterygium Detection in Anterior Segment Eye Images at Ophthalmological Medical Centers. International Journal of Online and Biomedical Engineering (iJOE), 20(8).
<https://online-journals.org/index.php/i-joe/article/view/48421/14973>
- Vue.js. (s.f.). The Progressive JavaScript Framework. <https://vuejs.org/>